

A-Low-Cost-Edge-AI-System-for-Automated-Waste-Classification-Using-ESP32-CAM

Varun Shankar

Sanat Vasisht

School of Electronics Engineering

School of Electronics Engineering

Vellore Institute of Technology

Vellore Institute of Technology

Vellore, Tamil Nadu

Vellore, Tamil Nadu

varun.shankar2024@vitstudent.ac.in

sanat.vasisht2024@vitstudent.ac.in

Abstract - The project showcases a Smart Waste Management System which makes use of the ESP32 Camera with in built AI system. Using this AI system, the system automatically classifies the waste materials into different categories in this project, we have taken biodegradable and non-biodegradable wastes. The classification is done with the help of a trained Edge Impulse ML model by taking various pictures of the test objects in various angles and lighting which get processed by the ML model and finally, trains the system to identify objects based on the category. After classification, with the help of a servo motor, the top of the system rotates to either side depending on the object type. The type of waste and other details of the object are displayed using an OLED display. This system, utilizes AI, image processing and microcontroller basics for efficient waster segregation and also helps in improving accuracy and reducing manual effort while also being cost effective, compact and environment friendly.

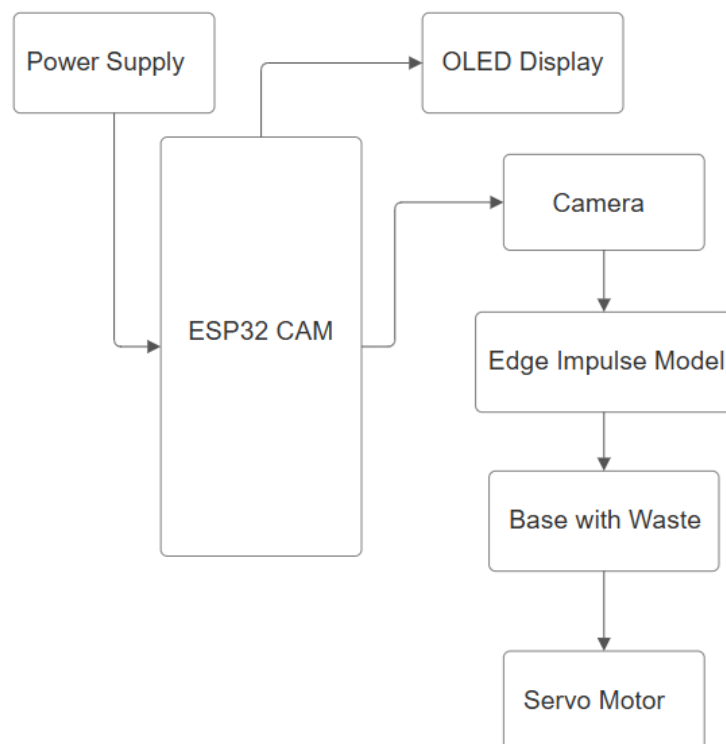
I. Literature Review

Year	Title of the Paper	Methodology	Key Finding	Limitations
2025	Optimizing Waste Classification using AI	Waste segregation performed using CNN,ESP32 Cam and moisture sensor. Arduino is main control unit	Reduced manual effort and increased efficiency. Better segregation categories like dry, wet and paper	High computation required and is difficult to adapt to different waste types
2024	Automatic Waste Sorting Machine	Electromechanical System which uses microcontroller, sensors and AI along with ML for sorting objects into metal, plastic ,paper and glass	Better recyclability and more better environmental sustainability.	Limited AI use and cant handle complex waste properly
2025	A High Efficient and Cost	Real time segregation using CNN,ESP32 Cam, metal and gas sensors	High accuracy(97-98%), very less misclassification,	Complex system and high cost of sensors.

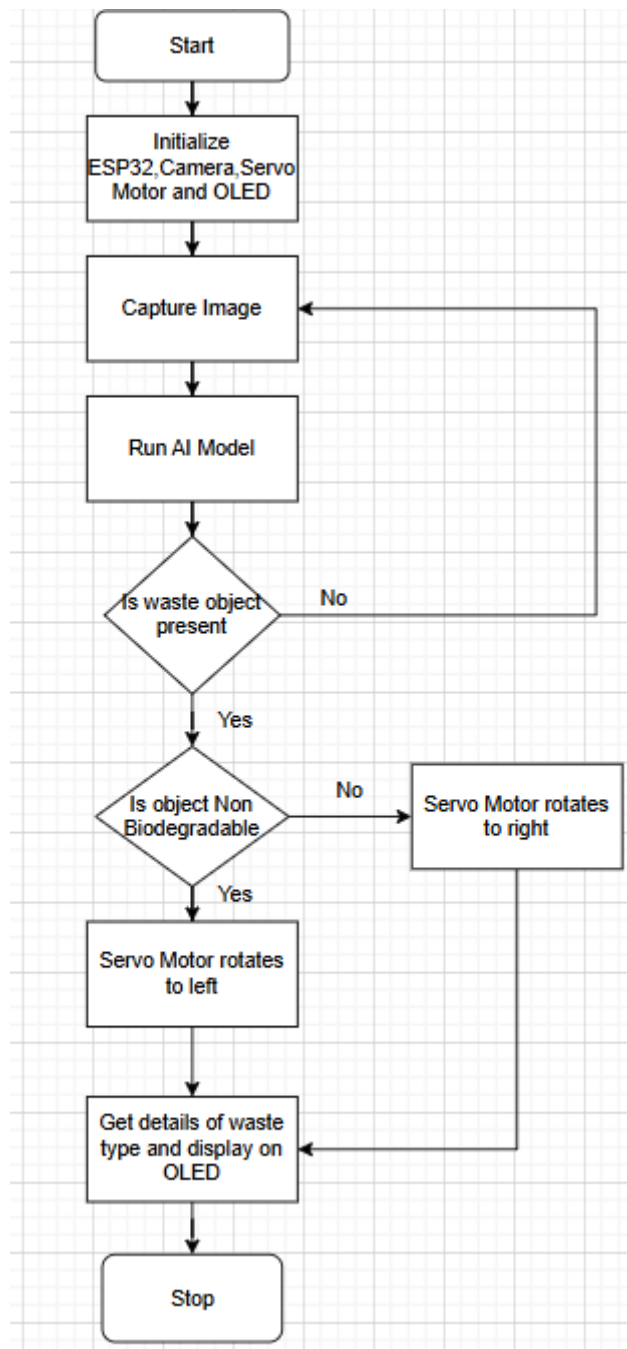
	Effective Waste Sorting Solution using Advanced Sensors and AI		also supports hazardous waste segregation	Depends too much on hardware components.
2023	Automated Smart Dustbin using ESP32	Uses Ultrasonic sensors ,PIR sensors and GSM	Automatic IOT notifications along with waste segregation into wet and dry	Limited classification only into wet and dry
2024	Smart Bin: A Waste Classification System for Eco-Friendly Urban Environments	Uses ESP32 Cam for image taking and uses OpenCV and TensorFlow as ML base	Classifies the waste into 6 types: paper ,plastic ,glass ,metal food and e-waste .Also enables automated report through SMS and allows touch free disposal	Slow due to low FPS and requires external devices like laptop for computation and relies on internet.

II. Methodology

Block Diagram



Flowchart



Code

```
#include <finalmpmcproj_inferencing.h>
#include "edge-impulse-sdk/dsp/image/image.hpp"

#include "esp_camera.h"
#include <Wire.h>
#include <Adafruit_GFX.h>
```

```

#include <Adafruit_SSD1306.h>

#include <ESP32Servo.h> // Include ESP32Servo library

Servo myServo; // Create a Servo object

unsigned long lastActionTime = 0;

const unsigned long delayTime = 5000; // 5 seconds

bool canMove = true;

// Select camera model - find more camera models in camera_pins.h file here
// https://github.com/espressif/arduino-esp32/blob/master/libraries/ESP32/examples/Camera/CameraWebServer/camera\_pins.h

// #define CAMERA_MODEL_ESP_EYE // Has PSRAM
#define CAMERA_MODEL_AI_THINKER // Has PSRAM

#if defined(CAMERA_MODEL_ESP_EYE)
#define PWDN_GPIO_NUM  -1
#define RESET_GPIO_NUM  -1
#define XCLK_GPIO_NUM  4
#define SIOD_GPIO_NUM  18
#define SIOC_GPIO_NUM  23

#define Y9_GPIO_NUM  36
#define Y8_GPIO_NUM  37
#define Y7_GPIO_NUM  38
#define Y6_GPIO_NUM  39
#define Y5_GPIO_NUM  35
#define Y4_GPIO_NUM  14
#define Y3_GPIO_NUM  13
#define Y2_GPIO_NUM  34
#define VSYNC_GPIO_NUM  5
#define HREF_GPIO_NUM  27

```

```

#define PCLK_GPIO_NUM 25

#elif defined(CAMERA_MODEL_AI_THINKER)
#define PWDN_GPIO_NUM 32
#define RESET_GPIO_NUM -1
#define XCLK_GPIO_NUM 0
#define SIOD_GPIO_NUM 26
#define SIOC_GPIO_NUM 27

#define Y9_GPIO_NUM 35
#define Y8_GPIO_NUM 34
#define Y7_GPIO_NUM 39
#define Y6_GPIO_NUM 36
#define Y5_GPIO_NUM 21
#define Y4_GPIO_NUM 19
#define Y3_GPIO_NUM 18
#define Y2_GPIO_NUM 5
#define VSYNC_GPIO_NUM 25
#define HREF_GPIO_NUM 23
#define PCLK_GPIO_NUM 22

#else
#error "Camera model not selected"
#endif

/* Constant defines ----- */
#define EI_CAMERA_RAW_FRAME_BUFFER_COLS 320
#define EI_CAMERA_RAW_FRAME_BUFFER_ROWS 240
#define EI_CAMERA_FRAME_BYTE_SIZE 3

#define I2C_SDA 15
#define I2C_SCL 2

```

```

#define SERVO_PIN 12 // Define servo control pin

TwoWire I2Cbus = TwoWire(0);

// Display defines
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
#define SCREEN_ADDRESS 0x3C
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &I2Cbus, OLED_RESET);

/* Private variables ----- */

static bool debug_nn = false; // Set this to true to see e.g. features generated from the raw signal
static bool is_initialised = false;
uint8_t *snapshot_buf; //points to the output of the capture

static camera_config_t camera_config = {
    .pin_pwdn = PWDN_GPIO_NUM,
    .pin_reset = RESET_GPIO_NUM,
    .pin_xclk = XCLK_GPIO_NUM,
    .pin_sscb_sda = SIOD_GPIO_NUM,
    .pin_sscb_scl = SIOC_GPIO_NUM,

    .pin_d7 = Y9_GPIO_NUM,
    .pin_d6 = Y8_GPIO_NUM,
    .pin_d5 = Y7_GPIO_NUM,
    .pin_d4 = Y6_GPIO_NUM,
    .pin_d3 = Y5_GPIO_NUM,
    .pin_d2 = Y4_GPIO_NUM,
    .pin_d1 = Y3_GPIO_NUM,
    .pin_d0 = Y2_GPIO_NUM,
    .pin_vsync = VSYNC_GPIO_NUM,

```

```

.pin_href = HREF_GPIO_NUM,
.pin_pclk = PCLK_GPIO_NUM,

//XCLK 20MHz or 10MHz for OV2640 double FPS (Experimental)
.xclk_freq_hz = 20000000,
.ledc_timer = LEDC_TIMER_0,
.ledc_channel = LEDC_CHANNEL_0,

.pixel_format = PIXFORMAT_JPEG, //YUV422,GRAYSCALE,RGB565,JPEG
.frame_size = FRAMESIZE_QVGA, //QQVGA-UXGA Do not use sizes above QVGA when not
JPEG

.jpeg_quality = 12, //0-63 lower number means higher quality
.fb_count = 1, //if more than one, i2s runs in continuous mode. Use only with JPEG
.fb_location = CAMERA_FB_IN_PSRAM,
.grab_mode = CAMERA_GRAB_WHEN_EMPTY,
};

/* Function definitions ----- */
bool ei_camera_init(void);
void ei_camera_deinit(void);
bool ei_camera_capture(uint32_t img_width, uint32_t img_height, uint8_t *out_buf) ;
/* Function definitions ----- */

static int ei_camera_get_data(size_t offset, size_t length, float *out_ptr);
/**
 * @brief   Arduino setup function
 */
void setup()
{
    // put your setup code here, to run once:
    Serial.begin(115200);

```

```

// Initialize I2C with our defined pins
I2Cbus.begin(I2C_SDA, I2C_SCL, 100000);

// SSD1306_SWITCHCAPVCC = generate display voltage from 3.3V internally
if (!display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS)) {
    Serial.printf("SSD1306 OLED display failed to initialize.\nCheck that display SDA is connected
to pin %d and SCL connected to pin %d\n", I2C_SDA, I2C_SCL);
    while (true);
}

myServo.attach(SERVO_PIN);
myServo.write(90); // Set initial servo position to 90 degrees

//comment out the below line to start inference immediately after upload
while (!Serial);
Serial.println("Edge Impulse Inferencing Demo");
if (ei_camera_init() == false) {
    ei_printf("Failed to initialize Camera!\r\n");
}
else {
    ei_printf("Camera initialized\r\n");
}

ei_printf("\nStarting continious inference in 2 seconds...\n");
display.clearDisplay();
display.setCursor(0, 0);
display.setTextSize(1);
display.setTextColor(SSD1306_WHITE);
display.print("AI\n Smart waste system\n");
display.display();

ei_sleep(2000);

```



```

        display.clearDisplay();

        display.print("Starting continious\n inference in\n 2 seconds...");

        display.display();

    }

/**
 * @brief    Get data and run inferencing
 *
 * @param[in] debug  Get debug info if true
 */
void loop()
{

    display.clearDisplay();

    // instead of wait_ms, we'll wait on the signal, this allows threads to cancel us...
    if (ei_sleep(5) != EI_IMPULSE_OK) {
        return;
    }

    snapshot_buf = (uint8_t*)malloc(EI_CAMERA_RAW_FRAME_BUFFER_COLS *
EI_CAMERA_RAW_FRAME_BUFFER_ROWS * EI_CAMERA_FRAME_BYTE_SIZE);

    // check if allocation was successful
    if(snapshot_buf == nullptr) {
        ei_printf("ERR: Failed to allocate snapshot buffer!\n");
        return;
    }

    ei::signal_t signal;

    signal.total_length = EI_CLASSIFIER_INPUT_WIDTH * EI_CLASSIFIER_INPUT_HEIGHT;
    signal.get_data = &ei_camera_get_data;

```

```

    if (ei_camera_capture((size_t)EI_CLASSIFIER_INPUT_WIDTH,
(size_t)EI_CLASSIFIER_INPUT_HEIGHT, snapshot_buf) == false) {
        ei_printf("Failed to capture image\r\n");
        free(snapshot_buf);
        return;
    }

    // Run the classifier
    ei_impulse_result_t result = { 0 };

    EI_IMPULSE_ERROR err = run_classifier(&signal, &result, debug_nn);
    if (err != EI_IMPULSE_OK) {
        ei_printf("ERR: Failed to run classifier (%d)\n", err);
        return;
    }

    // print the predictions
    ei_printf("Predictions (DSP: %d ms., Classification: %d ms., Anomaly: %d ms.): \n",
        result.timing.dsp, result.timing.classification, result.timing.anomaly);

    #if EI_CLASSIFIER_OBJECT_DETECTION == 1
        ei_printf("Object detection bounding boxes:\r\n");
        for (uint32_t i = 0; i < result.bounding_boxes_count; i++) {
            ei_impulse_result_bounding_box_t bb = result.bounding_boxes[i];
            if (bb.value == 0) {
                continue;
            }
            ei_printf(" %s (%f) [ x: %u, y: %u, width: %u, height: %u ]\r\n",
                bb.label,
                bb.value,
                bb.x,
                bb.y,
                bb.width,

```

```

        bb.height);
display.setCursor(0, 20);
display.setTextSize(2);
display.setTextColor(SSD1306_WHITE);
display.print(bb.label);
display.print("-");
display.print(int((bb.value)*100));
display.print("%");
display.display();
unsigned long currentTime = millis();

// allow movement only after 5 sec delay
if (currentTime - lastActionTime >= delayTime) {
    canMove = true;
}

if (canMove) {
    if (bb.label == "battery") {
        myServo.write(180); // RIGHT
        delay(2000);
        myServo.write(90); // CENTER

        lastActionTime = millis();
        canMove = false;
    }

    else if (bb.label == "paper") {
        myServo.write(0); // LEFT
        delay(2000);
        myServo.write(90); // CENTER

        lastActionTime = millis();
    }
}

```

```

        canMove = false;
    }
}

}

// Print the prediction results (classification)
#else
    ei_printf("Predictions:\r\n");
    for (uint16_t i = 0; i < EI_CLASSIFIER_LABEL_COUNT; i++) {
        ei_printf(" %s: ", ei_classifier_inferencing_categories[i]);
        ei_printf("%.5f\r\n", result.classification[i].value);
    }
#endif

// Print anomaly result (if it exists)
#if EI_CLASSIFIER_HAS_ANOMALY
    ei_printf("Anomaly prediction: %.3f\r\n", result.anomaly);
#endif

#if EI_CLASSIFIER_HAS_VISUAL_ANOMALY
    ei_printf("Visual anomalies:\r\n");
    for (uint32_t i = 0; i < result.visual_ad_count; i++) {
        ei_impulse_result_bounding_box_t bb = result.visual_ad_grid_cells[i];
        if (bb.value == 0) {
            continue;
        }
        ei_printf(" %s (%f) [ x: %u, y: %u, width: %u, height: %u ]\r\n",
            bb.label,
            bb.value,
            bb.x,
            bb.y,
            bb.width,

```

```

        bb.height);
    }
#endif

    free(snapshot_buf);

}

/**
 * @brief Setup image sensor & start streaming
 *
 * @retval false if initialisation failed
 */
bool ei_camera_init(void) {

    if (is_initialised) return true;

#ifdef CAMERA_MODEL_ESP_EYE
    pinMode(13, INPUT_PULLUP);
    pinMode(14, INPUT_PULLUP);
#endif

    //initialize the camera
    esp_err_t err = esp_camera_init(&camera_config);
    if (err != ESP_OK) {
        Serial.printf("Camera init failed with error 0x%x\n", err);
        return false;
    }

    sensor_t * s = esp_camera_sensor_get();
    // initial sensors are flipped vertically and colors are a bit saturated
    if (s->id.PID == OV3660_PID) {

```

```

    s->set_vflip(s, 1); // flip it back

    s->set_brightness(s, 1); // up the brightness just a bit

    s->set_saturation(s, 0); // lower the saturation
}

#ifdef CAMERA_MODEL_M5STACK_WIDE
    s->set_vflip(s, 1);

    s->set_hmirror(s, 1);
#elif defined(CAMERA_MODEL_ESP_EYE)
    s->set_vflip(s, 1);

    s->set_hmirror(s, 1);

    s->set_awb_gain(s, 1);
#endif

    is_initialised = true;

    return true;
}

/**
 * @brief    Stop streaming of sensor data
 */
void ei_camera_deinit(void) {

    //deinitialize the camera
    esp_err_t err = esp_camera_deinit();

    if (err != ESP_OK)
    {
        ei_printf("Camera deinit failed\n");

        return;
    }
}

```

```

    is_initialised = false;

    return;
}

/**
 * @brief    Capture, rescale and crop image
 *
 * @param[in] img_width    width of output image
 * @param[in] img_height   height of output image
 * @param[in] out_buf      pointer to store output image, NULL may be used
 *                          if ei_camera_frame_buffer is to be used for capture and resize/cropping.
 *
 * @retval    false if not initialised, image captured, rescaled or cropped failed
 *
 */
bool ei_camera_capture(uint32_t img_width, uint32_t img_height, uint8_t *out_buf) {
    bool do_resize = false;

    if (!is_initialised) {
        ei_printf("ERR: Camera is not initialized\r\n");
        return false;
    }

    camera_fb_t *fb = esp_camera_fb_get();

    if (!fb) {
        ei_printf("Camera capture failed\n");
        return false;
    }

    bool converted = fmt2rgb888(fb->buf, fb->len, PIXFORMAT_JPEG, snapshot_buf);

```

```

esp_camera_fb_return(fb);

if(!converted){
    ei_printf("Conversion failed\n");
    return false;
}

if ((img_width != EI_CAMERA_RAW_FRAME_BUFFER_COLS)
    || (img_height != EI_CAMERA_RAW_FRAME_BUFFER_ROWS)) {
    do_resize = true;
}

if (do_resize) {
    ei::image::processing::crop_and_interpolate_rgb888(
        out_buf,
        EI_CAMERA_RAW_FRAME_BUFFER_COLS,
        EI_CAMERA_RAW_FRAME_BUFFER_ROWS,
        out_buf,
        img_width,
        img_height);
}

return true;
}

static int ei_camera_get_data(size_t offset, size_t length, float *out_ptr)
{
    // we already have a RGB888 buffer, so recalculate offset into pixel index
    size_t pixel_ix = offset * 3;
    size_t pixels_left = length;
    size_t out_ptr_ix = 0;

```



```

while (pixels_left != 0) {
    // Swap BGR to RGB here
    // due to https://github.com/espressif/esp32-camera/issues/379
    out_ptr[out_ptr_ix] = (snapshot_buf[pixel_ix + 2] << 16) + (snapshot_buf[pixel_ix + 1] << 8) +
    snapshot_buf[pixel_ix];

    // go to the next pixel
    out_ptr_ix++;
    pixel_ix+=3;
    pixels_left--;
}
// and done!
display.clearDisplay();
return 0;
}

```

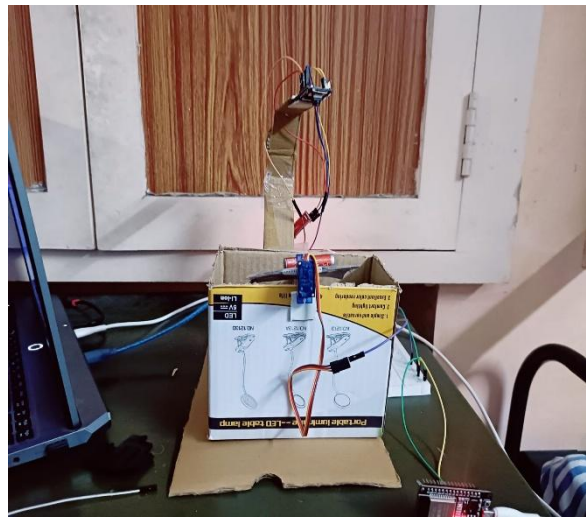
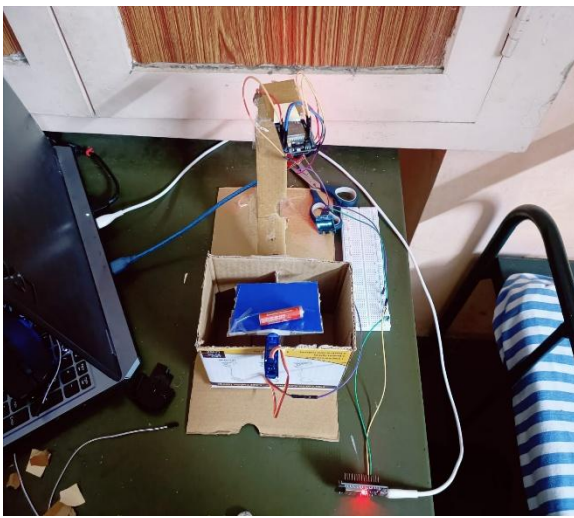
```

#ifndef EI_CLASSIFIER_SENSOR || EI_CLASSIFIER_SENSOR !=
EI_CLASSIFIER_SENSOR_CAMERA
#error "Invalid model for current sensor"
#endif

```

III. Result

The system compiled and ran successfully, showing the type of waste kept on the system and segregating it into the two columns using the servo motor. The ESP32 Camera accurately identified the waste type and segregated it into its type with the help of the Edge Impulse model trained using images. The setup was tested using two objects a battery for non-biodegradable waste and a piece of paper for biodegradable waste. After providing the power, the system starts to capture images of the object kept in its focus. Based on the data collected, the servo motor rotated the paper to the right side and the battery to the left side and afterwards comes back to its normal position. The OLED display gave a visual feedback of the waste type and its confidence percentage. The accuracy of the system was calculated to be 90%. The results prove that the system is successful in performing automated classification along with efficient segregation at high accuracies.



System Setup



OLED Output

IV. Conclusion and Future Work

The system designed successfully demonstrates a smart waste segregation system with the help of the ESP32 Cam as well as edge impulse. The system advantages include the on-device object detection, the visual output via the OLED display and the automatic servo motor rotation for waste segregation. What makes this different from the normal waste management systems are its elimination of any external control while also providing a compact, low cost and efficient solution for the increasing level of untreated waste.

The future applications and changes for this project are:

- Increasing the different categories into which the waste needs to be classified like plastic, metal, glass, organic waste, etc. This shows that the system is easy to customize.
- By increasing the number of pictures taken of the waste object like in different angles and lighting, the overall accuracy of the system can further be increased.
- Like in real life, dustbins with three to four sections can be designed using this system by increasing the number of servo motors.
- IoT and ML integration by sending the data of the waste type and the usage of the bin. This further helps the system to understand which waste is being used more compared to others and how frequently the waste is produced.

V. References

- [1] I. Bhongale, S. Kendre, S. R. Nalamwar, A. Kakade, and S. Agawane, "Optimizing Waste Classification using AI," *Journal of Engineering and Technology Management*, vol. 76, pp. 762–764, 2025.
- [2] I. Sheikh, A. Tapase, P. Nagose, P. Bhalekar, and D. Raich, "Automatic Waste Sorting Machine," *International Journal of Advanced Research in Science, Communication and Technology (IJARSCT)*, vol. 4, no. 1, pp. 123–125, Apr. 2024.
- [3] K. E., T. Tamilvanan, B. Yuvarani, J. Vasanth, S. Sanjaykumar, and S. Manikandan, "A High Efficient and Cost Effective Waste Sorting Solution using Advanced Sensors and AI," in *Proc. International Conference on Trends in Material Science and Inventive Materials (ICTMIM)*, IEEE, 2025, pp. 673–675.
- [4] G. N. K. Murthy, A. S., A. K. N., Y. R., and J. T. S., "Automated Smart Dustbin Using ESP32," in *Proc. International Conference on Recent Advances in Science & Engineering Technology (ICRASET)*, IEEE, 2023.
- [5] V. Sathappan M., S. G. D., K. V., R. A., S. S., and K. E., "Smart Bin: A Waste Classification System for Eco-Friendly Urban Environments," in *2024 3rd IEEE International Conference on Artificial Intelligence for Internet of Things (AIIoT)*, 2024, pp. 1–6.